

COMPSCI 701:
Creating Maintainable Software
Lecture 2.1: Comprehensibility

Yu-Cheng Tu, Ewan Tempero
School of Computer Science

Agenda

- Agenda

- PMD
- Maintainability
- Motivation
- Comprehensibility
- PCM
- Motivation
- Key Points

- Admin
 - Assignment 1 (See Updates on Canvas)
 - Wrong signature in `Kalah.java` **major**
 - Wrong details in `README.md` (minor)
 - The tests specify the actual requirements
 - Linking GitHub accounts
 - Reminder: Office Hourse — Monday 3pm-4pm Building 302, room 303
 - Other?
- PMD
- Class Exercise — names
- Comprehensibility
- Comprehension models
- Class Exercise — comprehensibility evaluation
- Lecture preparation
 - Change Cases recording (Canvas)

PMD

- Agenda
- **PMD**
- Maintainability
- Motivation
- Comprehensibility
- PCM
- Motivation
- Key Points

- a **linter** is a tool that detects aspects of code believed to lead to future (often maintainability) problems
- PMD is one such linter
- PMD can be customised as to what aspects it checks
- A “ruleset” **pmd.xml** has been provided in the Assignment 1 repo
- This ruleset will be used for the marking of Assignment 1
- Instructions for how you can use PMD yourself are available in the repo.
- Linters (and PMD) will be a topic for later discussion

- Agenda
- PMD
- Maintainability
- Motivation
- Comprehensibility
- PCM
- Motivation
- Key Points

Course Definition for Maintainability

- The degree of **effectiveness and efficiency** with which **maintenance activities** can be carried out on the product. It includes these **independent** sub-attributes:
 - **Comprehensibility** (Analysability)[†]—Degree of effectiveness and efficiency with which the implementation can be understood in order to conduct maintenance activities with confidence
 - **Alterability** (Modifiability)[‡]—degree to which a product or system can be effectively and efficiently changed without introducing defects or degrading existing product quality.
 - **Testability** — degree of effectiveness and efficiency with which test criteria can be established for a system, product or component and tests can be performed to determine whether those criteria have been met.
- **Notes**
 - As much as possible, arguments about each sub-attribute should not refer to any of the others. E.g., an argument about alterability must assume everything necessary about the code is understood
 - There is no mention of “after delivery” in these definitions — “maintenance activities” can occur before delivery

- Agenda
- PMD
- Maintainability
- Motivation
- Comprehensibility
- PCM
- Motivation
- Key Points

Class Exercise

1. With your neighbour (or by yourself if you wish)...
2. Examine `NoughtsAndCrosses-02.1` on canvas (PDF and `.java`)
(Note: This implementation uses the `IO` interface that you will use for the assignment. Check out `IO.html` for documentation.)
3. Determine the suitability of the variable name `result` in this code.
 - It is an **acceptable** name? Why or why not?
 - Is it the **best** name? Why or why not?
4. Write your responses in the Lecture 2.1a discussion on Canvas.

Evaluating identifier quality

- Agenda
- PMD
- Maintainability
- Motivation
- Comprehensibility
- PCM
- Motivation
- Key Points

- Given two choices we can usually make a convincing argument as to which is better to aid comprehensibility
- But
 - not always — sometimes it's not clear why one is better than the other
 - we have to construct brand new arguments every time — sometimes difficult to do and is not very efficient
- Writing code involves making decisions between multiple alternatives
- what is needed is a systematic process for determining which decision is better for comprehensibility (and alterability, testability, and so maintainability)

Comprehensibility

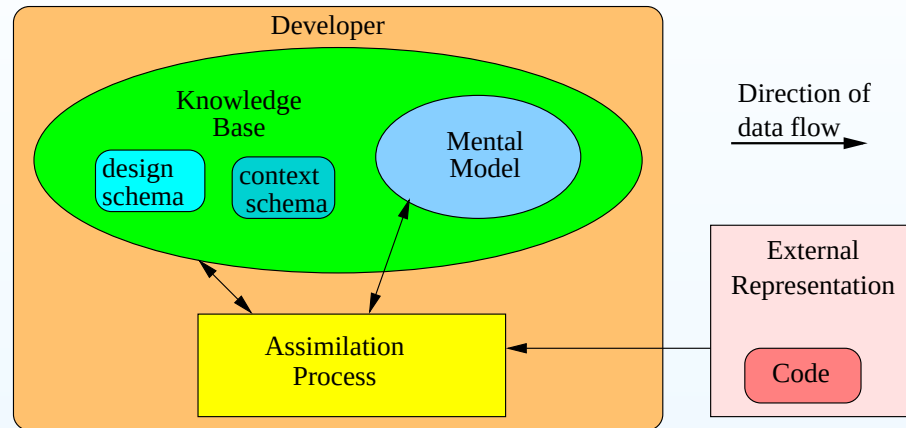
- Agenda
- PMD
- Maintainability
- Motivation
- **Comprehensibility**
- PCM
- Motivation
- Key Points

- Remember “Degree of effectiveness and efficiency with which the implementation can be understood in order to conduct maintenance activities with confidence”
- The more time and effort you need to put in to understand code **well enough** that you are confident enough to conduct maintenance activities the less comprehensible it is, therefore the less maintainable it is
- Therefore to increase the maintainability of our code we need to make it easier/faster/less costly to understand it **well enough**
- **how do we know we have increased the comprehensibility of the code?**
 - what does it mean to “comprehend” code (or anything)?

- Agenda
- PMD
- Maintainability
- Motivation
- Comprehensibility
- **PCM**
- Motivation
- Key Points

Program Comprehension Model

- Michael P. O'Brien *Software Comprehension—A Review and Research Direction* TR UL-CSIS-03-3, University of Limeric, 2003.



External representation Anything external to the developer E.g. source code, documentation, other developers, program traces, other relevant source code

Knowledge Base What the developer knows that is likely to be relevant, e.g. domain knowledge, knowledge of programming language,

Mental Model The developer's current understanding of the program under consideration, e.g. known parts and current hypotheses

Assimilation Process How the developer uses all available information to update the mental model, e.g. generation and verification of hypotheses

(Not all of this is described by O'Brien)

- Agenda
- PMD
- Maintainability
- Motivation
- Comprehensibility
- **PCM**
- Motivation
- Key Points

Example Using the PCM

- `result` is first used in lines 44–69. What “meaning” does it have for this section of code?
- `result` is next used in lines 83–120. What “meaning” does it have for this section of code? Is the same meaning as previously?
- When encountering `result` in the first section of code, the developer updates their Mental Model (MM) with what they have determined as its meaning.
- When encountering `result` in the second section of code, the developer looks it up in the MM (“I remember seeing this before so what did it mean then?”) and finds the first meaning there. When reading the second section of code
 1. they realise that the original meaning no longer is correct and so have to start again.

Wasted time $\Rightarrow$ less efficient
 2. they proceed under the assumption the original meaning is still true, and so misunderstand what the second section of code is doing

Mistake $\Rightarrow$ less effective

- Agenda
- PMD
- Maintainability
- Motivation
- Comprehensibility
- PCM
- Motivation
- Key Points

Class Exercise

1. Examine `NoughtsAndCrosses-01.java` on canvas (PDF and .java)
2. Answer the following questions, and write responses in Lecture 2.1b discussion on Canvas.
 - (a) Suggest a change in how the code is **formatted** that would improve comprehensibility and use the program comprehension model to justify your answer.
 - (b) Suggest at least 5 (FIVE) short (max 10 words) comments that could be added that would improve comprehensibility. Justify your choice using the program comprehension model.
 - (c) Identify the 3 (THREE) **biggest** comprehensibility problems. In each case, use the program comprehension model to justify your choice.

Determining Comprehensibility

- Agenda
- PMD
- Maintainability
- Motivation
- Comprehensibility
- PCM
- Motivation
- Key Points

- With experience, we learn characteristics of code that probably make it harder or easier to understand
- Some of this experience has been “codified” in various ways, some of which we will look at
- Examples: “Code Style”, “Clean code”, “Code smells” (all cover maintainability, not just comprehensibility)

- Agenda
- PMD
- Maintainability
- Motivation
- Comprehensibility
- PCM
- Motivation
- Key Points

Key Points

- We need systematic means to help make decisions to make our code maintainable
- We try to describe comprehension using comprehension models
- Comprehensibility depends on what is in KB, what is available in the ER, and/or how efficient the assimilation process is
- The “O’Brien” comprehension model may not be the most sophisticated but it will work for our purposes in this course